

ASSIGNMENT-01

MOVIE TICKET BOOKING SYSTEM

Course: B.Tech. CSE Semester: 05 Subject: System Design (TCS-504)

Submitted by: Anushka Choudhary Section: C1 Roll no: 13

Requirement Analysis:

Functional Requirements

FR1- Display movies: The system shall display all currently playing movies with their title, language and duration.

FR2- Display show: The system shall display all available shows for a selected movie, including screen number and start time.

FR3- Display seat layout: The system shall display the seat layout of a selected show and indicate whether each seat is available or booked.

FR4- Book seats: The system shall allow a customer to select one or more available seats and reject seats that are already booked.

FR5- Calculate ticket price: The system shall calculate the total ticket price according to seat type silver = ₹150, gold = ₹250, platinum = ₹400.

FR6- Make payment: The system shall allow payment using upi, card, cash.

FR7- Print ticket: The system shall print a ticket containing booking id, movie, screen, time, seat numbers and total amount.

FR8- Cancel booking: The system shall allow a confirmed booking to be cancelled and make its seats available again.

Non-Functional Requirements

NRF1- Modularity: Each major class shall be implemented separately, with one class per file.

NRF2- Extensibility: The payment system shall allow a new payment method such as NetBanking to be added without modifying BookingService.

NRF3- Input validation: Invalid menu choices and invalid seat numbers shall be handled with clear error messages without crashing the program.

NRF4- Maintainability: Each class and method shall have a single clear responsibility and avoid unnecessary duplication.

Noun Verb Analysis:

Noun	Class?	Reason
Movie	Yes	Has title, language and duration
Cinema	Yes	Represents the theatre
Screen	Yes	Represents one auditorium
Seat	Yes	Has seat number and type
Show	Yes	Represents one movie screening
ShowSeat	Yes	Stores seat status for a specific show
Customer	Yes	Stores customer details
Booking	Yes	Represents a reservation
Payment	Yes	Represents payment behaviour
Ticket	No	Ticket is output, handled by TicketPrinter
Seat Layout	No	It is only a display of ShowSeats

Important verbs/methods: listMovies(), listShows(), displaySeats(), bookSeat(), cancelSeat(), calculate(), pay(), printTicket(), cancelBooking().

Class Responsibilities:

Class	Knows	Does	Must NOT do
Movie	Knows title, language and duration.	Provides movie information.	Must not handle booking or payment.
Seat	Knows seat number and type.	Provides seat information.	Must not know show-specific booking status.
Screen	Knows screen number and owns seats.	Provides seats and screen information.	Must not calculate prices.
Cinema	Knows cinema name and owns screens.	Provides screens.	Must not process payments.

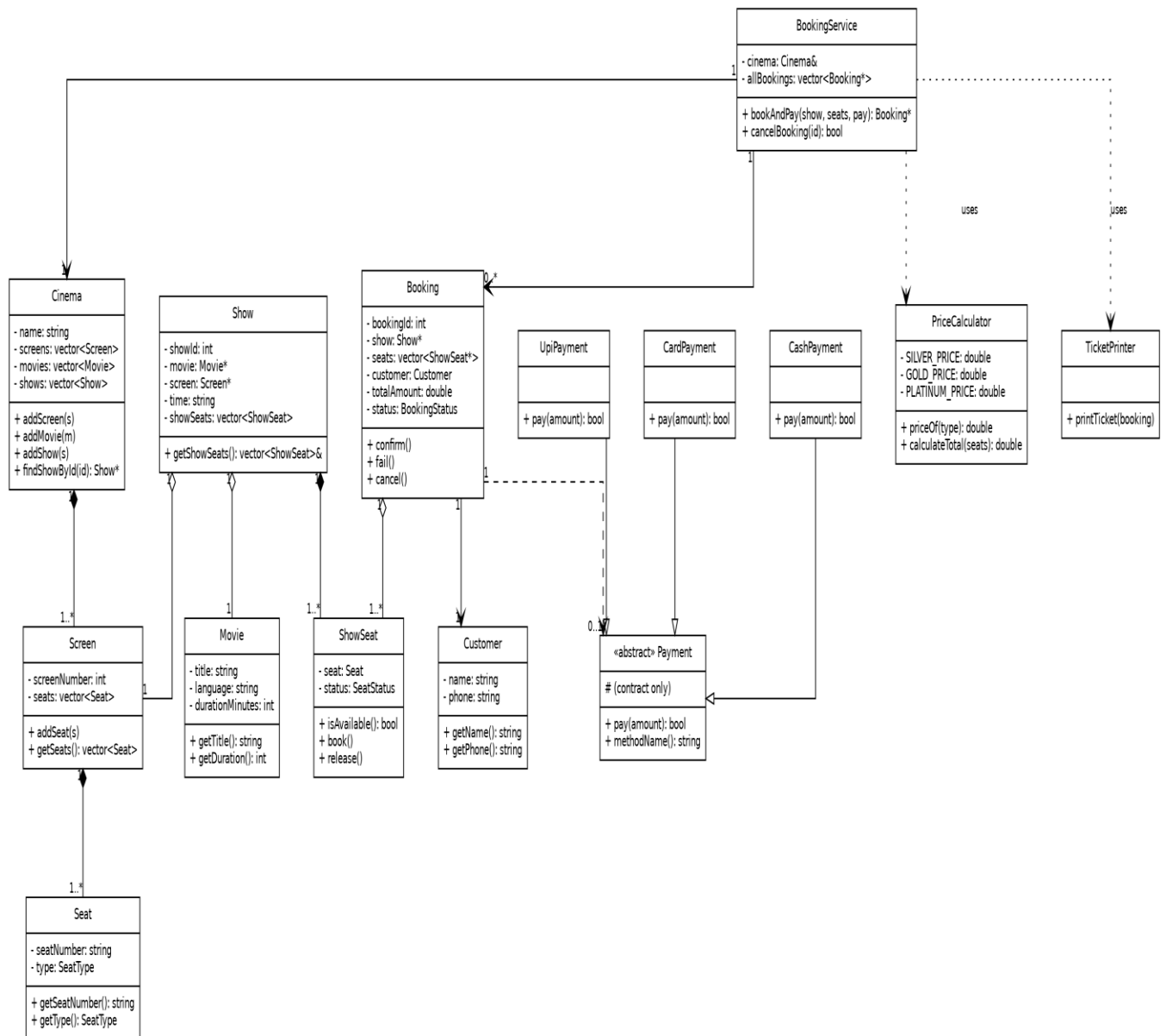
Show	Knows movie, screen, time and ShowSeats.	Displays/finds seats for that show.	Must not process payment.
ShowSeat	Knows a Seat and its status for one Show.	Books/cancels a seat and reports availability.	Must not calculate total booking price.
Customer	Knows name and phone.	Provides customer information.	Must not manage cinema inventory.
Booking	Knows booking ID, customer, show, seats, amount and status.	Stores booking state and cancellation.	Must not print tickets or perform payment.
Payment	Defines pay(amount).	Provides payment contract.	Must not contain booking orchestration.
UpiPayment / CardPayment / CashPayment	Know how their payment method works.	Perform payment.	Must not manage seats or tickets.
PriceCalculator	Knows seat-type prices.	Calculates total amount.	Must not book or print.
TicketPrinter	Knows ticket formatting.	Prints ticket.	Must not change booking state.
BookingService	Coordinates booking operations.	Runs booking flow and cancellation.	Must not own UI/menu input logic.
Main/Menu	Knows console interaction.	Reads input and invokes services.	Must not contain domain/business rules.

Relationship Table:

Pair	Relationship	Lifetime justification
Cinema — Screen	Composition	A cinema owns its screens; when the cinema is destroyed, its screens are part of that cinema and no longer exist in this model.
Screen — Seat	Composition	Seats are physical parts of a screen/auditorium in this

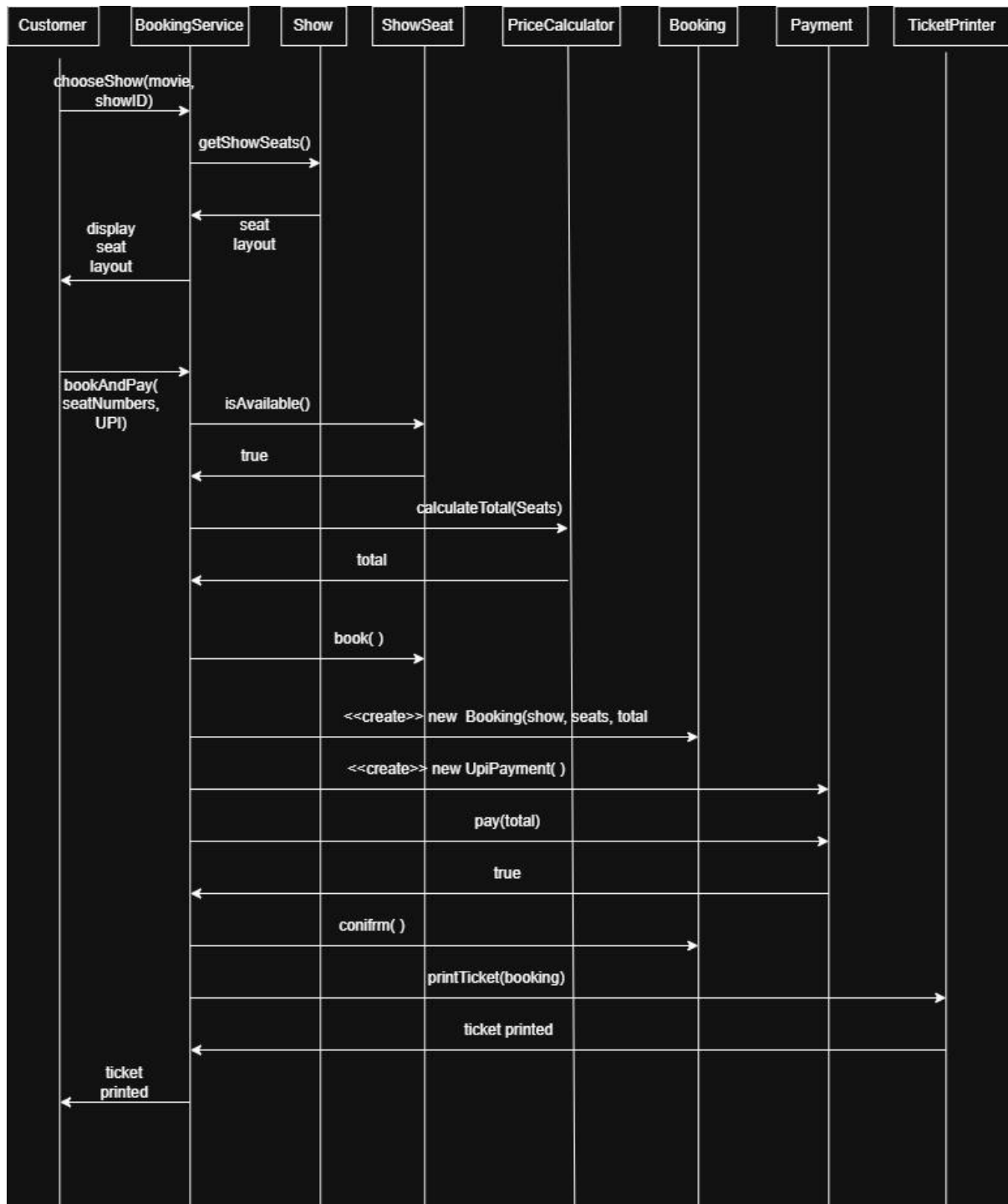
		model; their lifetime is owned by the Screen.
Show — Movie	Aggregation	A Movie exists independently and can be associated with multiple Shows.
Show — Screen	Aggregation	A Screen exists independently and can host multiple Shows at different times.
Show — ShowSeat	Composition	ShowSeats are created for a particular show and represent its seat statuses.
Booking — Customer	Association	The Customer exists independently of a particular booking.
Booking — ShowSeat	Aggregation	The ShowSeat belongs to the show and can exist even after a booking is cancelled.
Booking — Payment	Association	Payment is a service interaction used to complete the booking; the payment implementation is independent.
Payment — UpiPayment	Inheritance	UpiPayment is a type of Payment.
Payment — CardPayment	Inheritance	CardPayment is a type of Payment.
Payment — CashPayment	Inheritance	CashPayment is a type of Payment.
BookingService — Booking	Association	BookingService manages/creates bookings but bookings represent independent domain records.

Class Diagram:



Sequence Diagram:

Customer books 1 seat and pay by UPI



Demo Run:

Booking a seat.

```

● PS D:\projects\movieticket> g++ main.cpp
○ PS D:\projects\movieticket> ./a.exe
===== MOVIE TICKET BOOKING =====

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 1

  [1] 3 Idiots   Hindi   170 min
  [2] Interstellar English 169 min

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

  [1] 3 Idiots   Hindi   170 min
  [2] Interstellar English 169 min
Choose movie: 1
  [1] Screen-1   06:00 PM
  [2] Screen-2   09:00 PM
Choose show: 1

SCREEN-1 06:00 PM | 3 Idiots
SILVER   A1[ ] A2[ ] A3[ ] A4[ ]
GOLD    B1[ ] B2[ ] B3[ ]
PLATINUM C1[ ] C2[ ]

( [ ] = available [X] = booked )

Seats (e.g. A1,B2): A1,B2

Pay by: 1.UPI 2.Card 3.Cash > 1

A1 SILVER Rs.150
B2 GOLD Rs.250
TOTAL Rs.400
Processing UPI payment of Rs.400...
[UPI] Rs.400 paid successfully

```

```

===== TICKET =====
Booking ID : BK1001
Movie      : 3 Idiots
Screen     : Screen-1   06:00 PM
Seats      : A1, B2
Amount     : Rs.400      Status: CONFIRMED
=====

```

Edge cases exercised below:

(1) booking an already-booked seat is rejected with no state change.

```

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

  [1] 3 Idiots   Hindi   170 min
  [2] Interstellar English 169 min
Choose movie: 1
  [1] Screen-1   06:00 PM
  [2] Screen-2   09:00 PM
Choose show: 1

SCREEN-1 06:00 PM | 3 Idiots
SILVER   A1[X] A2[ ] A3[ ] A4[ ]
GOLD    B1[ ] B2[X] B3[ ]
PLATINUM C1[ ] C2[ ]

( [ ] = available [X] = booked )

Seats (e.g. A1,B2): A1,A2

Pay by: 1.UPI 2.Card 3.Cash > 1
Seat A1 is already BOOKED. Booking rejected, no seat changes state.

```

(2) an invalid seat number is rejected the same way

```
1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

[1] 3 Idiots   Hindi   170 min
[2] Interstellar English 169 min
Choose movie: 1
[1] Screen-1  06:00 PM
[2] Screen-2  09:00 PM
Choose show: 1

SCREEN-1  06:00 PM | 3 Idiots
SILVER      A1[X] A2[ ] A3[ ] A4[ ]
GOLD   B1[ ] B2[X] B3[ ]
PLATINUM   C1[ ] C2[ ]

( [ ] = available  [X] = booked )

Seats (e.g. A1,B2): A1,A2

Pay by: 1.UPI  2.Card  3.Cash > 1
Seat A1 is already BOOKED. Booking rejected, no seat changes state.

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

[1] 3 Idiots   Hindi   170 min
[2] Interstellar English 169 min
Choose movie: 1
[1] Screen-1  06:00 PM
[2] Screen-2  09:00 PM
Choose show: Z2
Invalid show choice.
```

(3) cancelling a confirmed booking releases its seats back to AVAILABLE

```
1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

[1] 3 Idiots   Hindi   170 min
[2] Interstellar English 169 min
Choose movie: 1
[1] Screen-1  06:00 PM
[2] Screen-2  09:00 PM
Choose show: Z2
Invalid show choice.

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 3
Enter Booking ID to cancel (e.g. 1001): BK1001
Booking BK1001 cancelled. Seats are now AVAILABLE again.

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2

[1] 3 Idiots   Hindi   170 min
[2] Interstellar English 169 min
Choose movie: 1
[1] Screen-1  06:00 PM
[2] Screen-2  09:00 PM
Choose show: 1

SCREEN-1  06:00 PM | 3 Idiots
SILVER      A1[ ] A2[ ] A3[ ] A4[ ]
GOLD   B1[ ] B2[ ] B3[ ]
PLATINUM   C1[ ] C2[ ]

( [ ] = available  [X] = booked )
```


(4) invalid menu input never crashes the program.

```
1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 9
    Invalid menu choice. Please try again.

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 7
    Invalid menu choice. Please try again.

1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: █
```

SOLID Principle applied

Principle	Where and Why
Single responsibility	Booking only holds booking data and status — it does NOT print tickets (TicketPrinter's job) and does NOT calculate price (PriceCalculator's job). A ticket-layout change and a pricing-rule change each touch exactly one class, never Booking.
Open/closed	Adding NetBanking needs only a new NetBankingPayment subclass implementing pay(). BookingService, Booking and every other existing class need zero edits, because they depend on the Payment abstraction, never on a concrete payment type.
Liskov Substitution	Every Payment subclass (UpiPayment, CardPayment, CashPayment) works anywhere a Payment* is expected, with no extra setup call beyond construction, and pay() always means “true = money collected” consistently across subtypes.
Dependency Inversion	BookingService depends on the abstract Payment type, not on UpiPayment/CardPayment/CashPayment directly — high-level booking logic is decoupled from low-level payment mechanics.

One thing deliberately NOT done:

We deliberately did not add a refund() method to the Payment interface, even though UpiPayment and CardPayment could plausibly support refunds while CashPayment (paid at the counter, in this simulation) cannot.

Interface Segregation says a class should not be forced to implement a method it has no meaningful behaviour for. Adding refund() to the base Payment contract would force CashPayment to either throw at runtime or fake an implementation that does nothing — both are worse than simply not having the method until it is a real requirement.

If refunds become necessary, the correct fix is a separate Refundable interface that only the payment types which actually support refunds implement, rather than bloating the shared Payment contract for every subtype.

Full source code

Complete listing of all 14 files, in the dependency order main.cpp includes them.

01_Movie.cpp

```
#include <string>

using namespace std;

// ===== Movie =====
// ONE responsibility: title, language, duration -- nothing else.
// Must NOT know about shows, screens, seats or pricing.
class Movie {
private:                // Encapsulation: fields hidden, only reachable via getters
    string title;
    string language;
    int durationMinutes;
public:
    Movie(string title, string language, int durationMinutes)
        : title(title), language(language), durationMinutes(durationMinutes) {}
    string getTitle() const { return title; }
    string getLanguage() const { return language; }
    int getDuration() const { return durationMinutes; }
};
```

02_Seat.cpp

```
#include <string>

using namespace std;

// SeatType is a property of the physical seat (SILVER/GOLD/PLATINUM row).
// Actual prices live in one place only: PriceCalculator (avoids magic numbers / duplication).
enum class SeatType { SILVER, GOLD, PLATINUM };
string seatTypeToString(SeatType type) {
    switch (type) {
        case SeatType::SILVER: return "SILVER";
        case SeatType::GOLD:   return "GOLD";
        case SeatType::PLATINUM: return "PLATINUM";
    }
}
```

```

    }
    return "UNKNOWN";
}

// ===== Seat =====
// ONE responsibility: one physical seat -- its number and type.
// Must NOT know whether it is booked (that belongs to ShowSeat, per-show).
class Seat {
private:          // Encapsulation
    string seatNumber;          // e.g. "A1"
    SeatType type;
public:
    Seat(string seatNumber, SeatType type) : seatNumber(seatNumber), type(type) {}
    string getSeatNumber() const { return seatNumber; }
    SeatType getType() const { return type; }
};

```

03_ShowSeat.cpp

```

#include <string>
using namespace std;
// depends on: Seat (02)
enum class SeatStatus { AVAILABLE, BOOKED };
// ===== ShowSeat =====
// ONE responsibility: the status of ONE seat FOR ONE show.
// WHY this class exists and not just Seat: physical seat A1 exists once on the screen,
// but its status is different for every show (booked for 6 PM, free for 9 PM).
// Status belongs to the Show, not to the physical chair.
// Must NOT change status except through book()/release() -- no direct field access from outside.
class ShowSeat {
private:          // Encapsulation
    Seat seat;          // this show's own copy of the seat's number/type
    SeatStatus status;
public:
    ShowSeat(Seat seat) : seat(seat), status(SeatStatus::AVAILABLE) {}

```

```

string getSeatNumber() const { return seat.getSeatNumber(); }
SeatType getType() const { return seat.getType(); }
SeatStatus getStatus() const { return status; }
bool isAvailable() const { return status == SeatStatus::AVAILABLE; }
void book() { status = SeatStatus::BOOKED; } // called by BookingService only
void release() { status = SeatStatus::AVAILABLE; } // used for cancellation & failed-payment
rollback
};

```

04_Screen.cpp

```

#include <vector>
#include <string>
using namespace std;
// depends on: Seat (02)
// ===== Screen =====
// ONE responsibility: one auditorium -- a screen number; owns its seats.
// Composition with Seat: if a Screen is destroyed, its Seats have no reason to exist independently.
class Screen {
private: // Encapsulation
    int screenNumber;
    vector<Seat> seats; // Composition: seats are created for, and belong to, this screen
public:
    Screen(int screenNumber) : screenNumber(screenNumber) {}
    void addSeat(Seat seat) { seats.push_back(seat); }
    int getScreenNumber() const { return screenNumber; }
    vector<Seat> getSeats() const { return seats; }
};

```

05_Show.cpp

```

#include <vector>
#include <string>
using namespace std;
// depends on: Movie (01), Seat (02), ShowSeat (03), Screen (04)

```

```

// ===== Show =====
// ONE responsibility: one screening = a Movie on a Screen at a time; owns its ShowSeats.
// Aggregation with Movie/Screen: a Show borrows them (they outlive/pre-exist the Show and
// are not deleted when the Show is). Composition with ShowSeat: the per-show seat states
// are created here and die with this Show.
// Must NOT print to the console (that is BookingService/TicketPrinter's job) and must NOT
// mutate seat status directly from outside its own ShowSeat objects.
class Show {
private:          // Encapsulation
    static int nextShowId;      // Static member: one shared id sequence for all shows
    int showId;
    Movie* movie;              // Aggregation
    Screen* screen;            // Aggregation
    string time;               // e.g. "06:00 PM"
    vector<ShowSeat> showSeats; // Composition
public:
    Show(Movie* movie, Screen* screen, string time)
        : movie(movie), screen(screen), time(time) {
        showId = nextShowId++;
        for (Seat s : screen->getSeats()) {
            showSeats.push_back(ShowSeat(s)); // «create» -- every seat starts AVAILABLE for this show
        }
    }
    int getShowId() const { return showId; }
    Movie* getMovie() const { return movie; }
    Screen* getScreen() const { return screen; }
    string getTime() const { return time; }
    vector<ShowSeat>& getShowSeats() { return showSeats; } // reference so callers can
    book()/release()
};
int Show::nextShowId = 1;

```

```

#include <vector>
#include <string>
using namespace std;

// depends on: Movie (01), Screen (04), Show (05)
// ===== Cinema =====
// ONE responsibility: the theatre -- its name; owns its screens. For this single-cinema
// system it is also the natural home for the movie catalog and the scheduled shows.
// Composition with Screen/Show: destroy the cinema and its screens/shows go with it.
// Must NOT contain booking, pricing or payment logic -- it is a container, not a service.
class Cinema {
private:                                // Encapsulation
    string name;
    vector<Screen> screens;             // Composition
    vector<Movie> movies;
    vector<Show> shows;                 // Composition
public:
    Cinema(string name) : name(name) {
        // reserve capacity up-front: Show stores Movie*/Screen* pointers into these vectors,
        // so we avoid pointer invalidation from reallocation when more entries are added.
        movies.reserve(10);
        screens.reserve(10);
        shows.reserve(20);
    }
    string getName() const { return name; }
    void addMovie(Movie movie) { movies.push_back(movie); }
    void addScreen(Screen screen) { screens.push_back(screen); }
    void addShow(Show show) { shows.push_back(show); }
    vector<Movie>& getMovies() { return movies; }
    vector<Screen>& getScreens() { return screens; }
    vector<Show>& getShows() { return shows; }
    // shows scheduled for a given movie (by title, since Movie has no id field)
    vector<Show*> showsForMovie(Movie* movie) {

```

```

    vector<Show*> result;

    for (auto& s : shows) if (s.getMovie() == movie) result.push_back(&s);

    return result;
}

Show* findShowById(int id) {
    for (auto& s : shows) if (s.getShowId() == id) return &s;

    return nullptr;
}
};

```

07_Customer.cpp

```

#include <string>

using namespace std;

// ===== Customer =====

// ONE responsibility: name and phone. Nothing about bookings or payments -- Customer only
// ASSOCIATES with BookingService to get things done; it does not own or drive the booking.

class Customer {
private:
    // Encapsulation
    string name;
    string phone;
public:
    Customer(string name, string phone) : name(name), phone(phone) {}

    Customer() : Customer("Guest", "NA") {} // Compile-time polymorphism: overloaded ctor,
    delegates via this()

    string getName() const { return name; }

    string getPhone() const { return phone; }
};

```

08_Payment.cpp

```

#include <string>

using namespace std;

// ===== Payment (abstract) =====

// Abstraction: defines the payment CONTRACT only -- pay(amount) and methodName().

// No concrete "how money moves" logic belongs here; that is each subclass's job.

```

```
// Liskov substitution: every Payment child must work correctly through a Payment*
// with no extra setup call needed beyond construction.
class Payment {
public:
    virtual bool pay(double amount) = 0;    // pure virtual -> makes this class abstract
    virtual string methodName() const = 0;
    virtual ~Payment() {}                  // virtual destructor: safe delete through base pointer
};
```

09_PaymentTypes.cpp

```
#include <iostream>
#include <string>
#include <cstdlib>
using namespace std;
// depends on: Payment (08)
const int SIMULATED_SUCCESS_RATE_PERCENT = 90; // no magic numbers scattered around
// Inheritance: UpiPayment / CardPayment / CashPayment each extend Payment.
// Runtime polymorphism: calling pay() through a Payment* / Payment& invokes the
// override that matches the object's ACTUAL type, decided at run time, not compile time.
// ===== UpiPayment =====
class UpiPayment : public Payment {
public:
    bool pay(double amount) override {
        cout << " Processing UPI payment of Rs." << amount << "...\\n";
        bool success = (rand() % 100) < SIMULATED_SUCCESS_RATE_PERCENT;
        if (success) cout << " [UPI] Rs." << amount << " paid successfully\\n";
        else      cout << " [UPI] Payment FAILED - please retry\\n";
        return success;
    }
    string methodName() const override { return "UPI"; }
};
// ===== CardPayment =====
```



```

class CardPayment : public Payment {
public:
    bool pay(double amount) override {
        cout << " Processing Card payment of Rs." << amount << "...\\n";
        bool success = (rand() % 100) < SIMULATED_SUCCESS_RATE_PERCENT;
        if (success) cout << " [Card] Rs." << amount << " paid successfully\\n";
        else      cout << " [Card] Payment FAILED - please retry\\n";
        return success;
    }
    string methodName() const override { return "Card"; }
};

// ===== CashPayment =====
// Cash paid in person at the counter -- modeled here as always succeeding.
class CashPayment : public Payment {
public:
    bool pay(double amount) override {
        cout << " Processing Cash payment of Rs." << amount << "...\\n";
        cout << " [Cash] Rs." << amount << " paid successfully\\n";
        return true;
    }
    string methodName() const override { return "Cash"; }
};

```

10_Booking.cpp

```

#include <vector>
#include <string>
using namespace std;
// depends on: Show (05), ShowSeat (03), Customer (07)
enum class BookingStatus { PENDING, CONFIRMED, FAILED, CANCELLED };
string bookingStatusToString(BookingStatus status) {
    switch (status) {
        case BookingStatus::CONFIRMED: return "CONFIRMED";
        case BookingStatus::FAILED:    return "FAILED";
    }
}

```

```

        case BookingStatus::CANCELLED: return "CANCELLED";
        default: return "PENDING";
    }
}

// ===== Booking =====
// ONE responsibility: booking id, which show, which seats, which customer, total amount, status.
// Must NOT calculate price (PriceCalculator's job) and must NOT print tickets (TicketPrinter's job).
// Aggregation with Show/ShowSeat/Customer: Booking refers to them, does not own their lifetime.
class Booking {
private: // Encapsulation
    static int nextBookingId; // Static member: one shared id sequence for all bookings
    int bookingId;
    Show* show; // Aggregation
    vector<ShowSeat*> seats; // Aggregation
    Customer customer;
    double totalAmount;
    BookingStatus status;
public:
    Booking(Show* show, vector<ShowSeat*> seats, double totalAmount, Customer customer =
Customer())
        : show(show), seats(seats), customer(customer),
          totalAmount(totalAmount), status(BookingStatus::PENDING) {
        bookingId = nextBookingId++;
    }
    int getBookingId() const { return bookingId; }
    Show* getShow() const { return show; }
    vector<ShowSeat*> getSeats() const { return seats; }
    Customer getCustomer() const { return customer; }
    double getTotalAmount() const { return totalAmount; }
    BookingStatus getStatus() const { return status; }
    void confirm() { status = BookingStatus::CONFIRMED; } // called only by BookingService
    void fail() { status = BookingStatus::FAILED; }
    void cancel() { status = BookingStatus::CANCELLED; }

```

```
};

int Booking::nextBookingId = 1001; // -> BK1001, BK1002, ...
```

11_PriceCalculator.cpp

```
#include <vector>

using namespace std;

// depends on: ShowSeat (03), SeatType (02)

// Constants, not magic numbers -- one source of truth for prices.
const double SILVER_PRICE = 150.0;
const double GOLD_PRICE = 250.0;
const double PLATINUM_PRICE = 400.0;

// ===== PriceCalculator =====

// ONE responsibility: turn a list of seats into a total amount.
// Pure calculation only -- no console I/O, no state changes, no side effects.

class PriceCalculator {
public:
    static double priceOf(SeatType type) {
        switch (type) {
            case SeatType::SILVER: return SILVER_PRICE;
            case SeatType::GOLD: return GOLD_PRICE;
            case SeatType::PLATINUM: return PLATINUM_PRICE;
        }
        return 0.0;
    }

    static double calculateTotal(const vector<ShowSeat*>& seats) {
        double total = 0.0;
        for (ShowSeat* s : seats) total += priceOf(s->getType());
        return total;
    }
};
```

12_TicketPrinter.cpp

```
#include <iostream>
```

```

#include <vector>

#include <string>

using namespace std;

// depends on: Booking (10), Show (05), Movie (01), Screen (04), ShowSeat (03)

// ===== TicketPrinter =====

// ONE responsibility: format and print a ticket -- PRINTING ONLY.

// Must NOT confirm, cancel, or otherwise change a booking's state; a function named

// printTicket() should have no side effects on the objects it reads (clean-code rule).

class TicketPrinter {

public:

    static void printTicket(Booking* booking) {

        Show* show = booking->getShow();

        vector<ShowSeat*> seats = booking->getSeats();

        cout << "\n ===== TICKET =====\n";

        cout << "   Booking ID : BK" << booking->getBookingId() << "\n";

        cout << "   Movie    : " << show->getMovie()->getTitle() << "\n";

        cout << "   Screen   : Screen-" << show->getScreen()->getScreenNumber()

            << " " << show->getTime() << "\n";

        cout << "   Seats    : ";

        for (size_t i = 0; i < seats.size(); i++) {

            cout << seats[i]->getSeatNumber();

            if (i + 1 < seats.size()) cout << ", ";

        }

        cout << "\n   Amount    : Rs." << booking->getTotalAmount()

            << "   Status: " << bookingStatusToString(booking->getStatus()) << "\n";

        cout << " =====\n";

    }

};

```

13_BookingService.cpp

```

#include <iostream>

#include <vector>

#include <string>

```

```

#include <sstream>

using namespace std;

// depends on: Cinema (06), Show (05), ShowSeat (03), Booking (10), Payment (08/09),
//          PriceCalculator (11), TicketPrinter (12)
// ===== BookingService =====
// ONE responsibility: the orchestrator -- runs the booking flow end to end (F1-F8),
// coordinating the entity classes below it. Association with Cinema: BookingService
// uses it but does not own it.
// Open/Closed: adding NetBanking only means adding a new Payment subclass; this class
// (and Booking, Show, etc.) needs zero edits because it depends on Payment (the abstraction),
// never on a concrete payment type directly.
class BookingService {
private:
    Cinema& cinema;          // Association (this=BookingService); doesn't own Cinema
    vector<Booking*> allBookings; // bookings made in this session (for cancel / "my tickets")

    ShowSeat* findShowSeat(Show* show, string seatNumber) {
        for (ShowSeat& s : show->getShowSeats()) {
            if (s.getSeatNumber() == seatNumber) return &s;
        }
        return nullptr;
    }

    void printSeatRow(Show* show, SeatType type, string label) {
        cout << " " << label << "\t";
        for (ShowSeat& s : show->getShowSeats()) {
            if (s.getType() == type) {
                cout << s.getSeatNumber() << (s.isAvailable() ? "[ ] " : "[X] ");
            }
        }
        cout << "\n";
    }

public:
    BookingService(Cinema& cinema) : cinema(cinema) {}

```

```

// ----- F1 -----
void listMovies() {
    vector<Movie>& movies = cinema.getMovies();

    cout << "\n";

    for (size_t i = 0; i < movies.size(); i++) {
        cout << " [" << (i + 1) << "] " << movies[i].getTitle()
            << "\t" << movies[i].getLanguage()
            << "\t" << movies[i].getDuration() << " min\n";
    }
}

// ----- F2 -----
vector<Show*> listShowsForMovie(int movieIndex1Based) {
    vector<Movie>& movies = cinema.getMovies();

    if (movieIndex1Based < 1 || movieIndex1Based > (int)movies.size()) {
        cout << " Invalid movie choice.\n";
        return {};
    }

    Movie* movie = &movies[movieIndex1Based - 1];
    vector<Show*> shows = cinema.showsForMovie(movie);

    for (size_t i = 0; i < shows.size(); i++) {
        cout << " [" << (i + 1) << "] Screen-" << shows[i]->getScreen()->getScreenNumber()
            << "\t" << shows[i]->getTime() << "\n";
    }

    return shows;
}

// ----- F3 -----
void printSeatLayout(Show* show) {
    cout << "\n SCREEN-" << show->getScreen()->getScreenNumber()
        << " " << show->getTime() << " | " << show->getMovie()->getTitle() << "\n";

    printSeatRow(show, SeatType::SILVER, "SILVER");
    printSeatRow(show, SeatType::GOLD, "GOLD");
    printSeatRow(show, SeatType::PLATINUM, "PLATINUM");
}

```

```

        cout << "\n ( [ ] = available  [X] = booked )\n";
    }

    // splits "A1,B2" into {"A1","B2"}
    vector<string> parseSeatNumbers(string input) {
        vector<string> result;
        stringstream ss(input);
        string token;
        while (getline(ss, token, ',')) {
            // trim whitespace
            while (!token.empty() && token.front() == ' ') token.erase(token.begin());
            while (!token.empty() && token.back() == ' ') token.pop_back();
            if (!token.empty()) result.push_back(token);
        }
        return result;
    }

    // ----- F4 + F5 + F6 + F7, orchestrated together -----
    // FR4: if ANY selected seat is invalid or already booked, whole booking is rejected,
    //     no seat changes state.
    // FR6: booking is confirmed only AFTER payment succeeds; a failed payment releases
    //     the seats and marks the booking FAILED.
    Booking* bookAndPay(Show* show, vector<string> seatNumbers, int paymentChoice) {
        if (seatNumbers.empty()) {
            cout << " No seats entered. Booking rejected.\n";
            return nullptr;
        }

        // Validate everything FIRST -- nothing is booked until we know the whole request is valid.
        vector<ShowSeat*> chosen;
        for (string num : seatNumbers) {
            ShowSeat* found = findShowSeat(show, num);
            if (found == nullptr) {
                cout << " Invalid seat number: " << num << ". Booking rejected, no seat changes state.\n";
            }
        }
    }

```

```

        return nullptr;
    }
    if (!found->isAvailable()) {
        cout << " Seat " << num << " is already BOOKED. Booking rejected, no seat changes
state.\n";
        return nullptr;
    }
    chosen.push_back(found);
}

// F5: price the booking
double total = PriceCalculator::calculateTotal(chosen);
cout << "\n";
for (ShowSeat* s : chosen) {
    cout << " " << s->getSeatNumber() << " " << seatTypeToString(s->getType())
        << " Rs." << PriceCalculator::priceOf(s->getType()) << "\n";
}
cout << " TOTAL      Rs." << total << "\n";
// Reserve seats tentatively; rolled back below if payment fails (FR6).
for (ShowSeat* s : chosen) s->book();

Booking* booking = new Booking(show, chosen, total); // «create»
allBookings.push_back(booking);
// F6: pay by UPI, Card or Cash
Payment* payment;
if (paymentChoice == 1) payment = new UpiPayment(); // «create»
else if (paymentChoice == 2) payment = new CardPayment(); // «create»
else payment = new CashPayment(); // «create»
bool success = payment->pay(total); // runtime polymorphism: exact override picked at
runtime
if (success) {
    booking->confirm();
    TicketPrinter::printTicket(booking); // F7
}

```



```

    } else {
        for (ShowSeat* s : chosen) s->release(); // FR6: release seats on failure
        booking->fail();
        cout << " Booking NOT confirmed. Seats released back to AVAILABLE.\n";
    }
    delete payment;
    return booking;
}

// ----- F8 -----
bool cancelBooking(int bookingId) {
    for (Booking* b : allBookings) {
        if (b->getBookingId() == bookingId) {
            if (b->getStatus() != BookingStatus::CONFIRMED) {
                cout << " Booking BK" << bookingId << " is not in CONFIRMED state, cannot cancel.\n";
                return false;
            }
            for (ShowSeat* s : b->getSeats()) s->release();
            b->cancel();
            cout << " Booking BK" << bookingId << " cancelled. Seats are now AVAILABLE again.\n";
            return true;
        }
    }
    cout << " No booking found with ID BK" << bookingId << ".\n";
    return false;
}

// "My tickets"
void listMyBookings() {
    bool any = false;
    for (Booking* b : allBookings) {
        if (b->getStatus() == BookingStatus::CONFIRMED) {
            any = true;
            cout << " BK" << b->getBookingId() << " " << b->getShow()->getMovie()->getTitle()

```

```

        << " " << b->getShow()->getTime() << " Rs." << b->getTotalAmount()
        << " " << bookingStatusToString(b->getStatus()) << "\n";
    }
}
if (!any) cout << " No confirmed bookings yet.\n";
}
};

```

main.cpp

```

#include <iostream>

#include <string>

#include <vector>

#include <limits>

#include <ctime>

#include <cstdlib>

using namespace std;

// ---- Course rule: "one class per file, no header files". Since plain classes with
// no header can't be split across separately-compiled translation units without
// redeclaring them, this project stays modular by keeping each class in its own .cpp
// file and having main.cpp #include them in dependency order. Build with:
// g++ main.cpp -o ticketbooking

#include "01_Movie.cpp"
#include "02_Seat.cpp"
#include "03_ShowSeat.cpp"
#include "04_Screen.cpp"
#include "05_Show.cpp"
#include "06_Cinema.cpp"
#include "07_Customer.cpp"
#include "08_Payment.cpp"
#include "09_PaymentTypes.cpp"
#include "10_Booking.cpp"
#include "11_PriceCalculator.cpp"
#include "12_TicketPrinter.cpp"

```

```
#include "13_BookingService.cpp"

// builds a Screen with rows A(SILVER x4), B(GOLD x3), C(PLATINUM x2) -- matches the demo layout
Screen makeStandardScreen(int screenNumber) {
    Screen screen(screenNumber);
    for (int i = 1; i <= 4; i++) screen.addSeat(Seat("A" + to_string(i), SeatType::SILVER));
    for (int i = 1; i <= 3; i++) screen.addSeat(Seat("B" + to_string(i), SeatType::GOLD));
    for (int i = 1; i <= 2; i++) screen.addSeat(Seat("C" + to_string(i), SeatType::PLATINUM));
    return screen;
}
```

```
Cinema setupCinema() {
    Cinema cinema("Downtown Cinema");
    cinema.addMovie(Movie("3 Idiots", "Hindi", 170));
    cinema.addMovie(Movie("Interstellar", "English", 169));
    cinema.addScreen(makeStandardScreen(1));
    cinema.addScreen(makeStandardScreen(2));
    vector<Movie>& movies = cinema.getMovies();
    vector<Screen>& screens = cinema.getScreens();
    cinema.addShow(Show(&movies[0], &screens[0], "06:00 PM")); // 3 Idiots @ Screen-1
    cinema.addShow(Show(&movies[0], &screens[1], "09:00 PM")); // 3 Idiots @ Screen-2
    cinema.addShow(Show(&movies[1], &screens[0], "09:30 PM")); // Interstellar @ Screen-1
    return cinema;
}
```

// reads an integer choice safely; returns -1 on non-numeric input (edge case: no crash)

```
int readIntChoice() {
    string line;
    getline(cin, line);
    try {
        return stoi(line);
    } catch (...) {
        return -1;
    }
}
```

```

    }
}

void handleBookFlow(BookingService& service) {
    service.listMovies();
    cout << "Choose movie: ";
    int movieChoice = readIntChoice();
    vector<Show*> shows = service.listShowsForMovie(movieChoice);
    if (shows.empty()) {
        cout << " No shows available for that choice.\n";
        return;
    }
    cout << "Choose show: ";
    int showChoice = readIntChoice();
    if (showChoice < 1 || showChoice > (int)shows.size()) {
        cout << " Invalid show choice.\n";
        return;
    }
    Show* show = shows[showChoice - 1];
    service.printSeatLayout(show);
    cout << "\nSeats (e.g. A1,B2): ";
    string seatInput;
    getline(cin, seatInput);
    vector<string> seatNumbers = service.parseSeatNumbers(seatInput);
    cout << "\nPay by: 1.UPI 2.Card 3.Cash > ";
    int payChoice = readIntChoice();
    if (payChoice < 1 || payChoice > 3) {
        cout << " Invalid payment choice. Booking cancelled.\n";
        return;
    }
    service.bookAndPay(show, seatNumbers, payChoice);
}

void handleCancelFlow(BookingService& service) {

```

```

    cout << "Enter Booking ID to cancel (e.g. 1001): BK";
    int id = readIntChoice();
    if (id < 0) {
        cout << " Invalid booking id.\n";
        return;
    }
    service.cancelBooking(id);
}

int main() {
    srand((unsigned)time(nullptr)); // seed for simulated payment success/failure

    Cinema cinema = setupCinema();
    BookingService service(cinema);
    cout << "===== MOVIE TICKET BOOKING =====\n"
    bool running = true;
    while (running) {
        cout << "\n1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit\nChoose: ";
        int choice = readIntChoice();
        switch (choice) {
            case 1:
                service.listMovies();
                break;
            case 2:
                handleBookFlow(service);
                break;
            case 3:
                handleCancelFlow(service);
                break;
            case 4:
                service.listMyBookings();
                break;
            case 0:

```

```
    running = false;

    cout << "Goodbye!\n";

    break;

default:

    cout << " Invalid menu choice. Please try again.\n"; // edge case: no crash

}

}

return 0;

}
```